

7. MVP / Post-MVP

7.1 Delivery sequencing

7.1.1 Phase 1 — Plan CRUD (no execution). Entity model, Event Store + Outbox, Command Handlers for `PlanCreated` / `PlanEdited`, Projections, and `POST/PUT/GET /plans`. Fully self-contained and testable without Scheduler or Trading/Orders — ships first since nothing else has anything to act on until plans exist.

7.1.2 Phase 2 — Scheduled execution pipeline. Scheduler, Dispatcher, `execution-triggers`, Initiator, `InitiateExecution` with its idempotency key (2.5), `order-commands` publishing, plus the execution-viewing endpoints. Has a hard external dependency: can't be integration-tested until Trading/Orders confirms the topic and reason-code questions (5.3 #1-#2) — worth starting that conversation before development, not after.

7.1.3 Phase 3 — Closing the loop. Order-Status Consumer, `UpdateOrderStatus`, the `ExecutionProcessed` / `PlanDeactivated` branch (2.10), and the observability signals from `6-non-functional-requirements.md` 6.4. Still MVP, not a follow-on — without it, executions never resolve past `Initiated`, and a silent full-day dispatch failure (4.1) would be undetectable in production.

Idempotency (2.5) and the reactive `PlanDeactivated` path are core correctness properties bundled into these phases, not separable features to cut.

7.2 MVP API surface

- `POST /plans` — Phase 1
- `PUT /plans/{id}` — Phase 1
- `GET /plans` — Phase 1
- `GET /plans/{id}` — Phase 1
- `GET /plans/{id}/executions` — Phase 2 (empty until it ships)
- `GET /plans/{id}/executions/{id}` — Phase 2

All six ship as part of MVP — the brief's full API list is small enough that nothing is held back to post-MVP; the phasing above is about build order and testability, not permanent exclusion.

7.3 Explicitly post-MVP (deferred, with rationale)

1. **Proactive asset-availability reconciliation** (5.2 #6) — reactive detection (Phase 3) covers the same problem far cheaper; proactive reconciliation is undesigned scope with its own open questions.
2. **Manual re-execution / idempotency-override tooling** (4.4, 5.2 #5) — no confirmed support workflow yet; a direct, audited manual intervention is an acceptable stopgap.
3. **Read/write service split** (2.1) — internal CQRS separation already gives the seams to split later; premature at current load.
4. **Dispatch checkpoint/cursor** (vs. full re-scan, 3.5) — full re-scan is correct and cheap at current scale; only worth the complexity if re-scan cost becomes real.
5. **Outbox-style staging for the Execution Dispatcher** (vs. direct-produce, 2.2's exception / 3.5) — direct-produce leaves the Dispatcher's publish dependent on Kafka being available at that moment rather than decoupled from it; revisit only if that dependency causes real operational pain, since staging costs a DB write and a Debezium hop per due plan.
6. **Order-Status Consumer horizontal scale-out** (4.6) — blocked on confirming `order-data`'s partitioning with Trading/Orders (5.3 #3) anyway.
7. **Optimistic concurrency on Plan edits** (5.3 #4) — no confirmed requirement yet; last-write-wins is an acceptable MVP default.
8. **Dedicated plan-notifications topic** (2.11) — direct subscription to `investment-plan-data` is strictly less work; revisit only if schema coupling becomes a real problem.

Push-notification *delivery* itself depends on whether a Notification Domain capable of consuming our events already exists (5.2 #4) — a dependency on another team's system, not something this timeline can guarantee. Our MVP scope is publishing events rich enough for it to work (2.11), covered in Phase 3.